

第8章 网络通信

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

本章学习目标

- 回顾 TCP/UDP、端口、字节序,理解 Node 如何封装 C 的 socket API
- 掌握 net 模块:TCP 服务端与客户端完整示例
- 掌握 dgram 模块:UDP 无连接通信
- 掌握超时设置与"为什么必须设置超时"的生产教训
- 掌握 fetch 发 HTTP 请求:GET/POST/JSON 完整示例
- 掌握 http.createServer 最小 HTTP 服务端
- 理解粘包/拆包问题与长度前缀协议设计,掌握优雅关闭

从 C 到 JavaScript:本章主题如何迁移

C 的 socket 编程是"仪式感之王":socket()、bind()、listen()、accept()、connect()、send()、recv(),每个都返回错误码,还得用 htons/htonl 转换字节序,处理 EINTR 重试,管理 fd 生命周期。Node 把这些全部封装成简洁的对象:net.Server/net.Socket/dgram.Socket,事件驱动,数据到达即触发 'data' 事件,不需要手动 accept 循环——事件循环替你完成了 accept。

最大的心智差异依然是 异步 + 事件 :C 的 recv 阻塞等数据;Node 的 socket.on('data') 注册回调,数据到了事件循环调你。读写都不阻塞,一个进程可以同时处理成千上万连接(第9章细讲)。

字节序方面,Node 帮你藏掉了大部分:TCP 流无边界、无字节序问题(字节原样),Buffer 的 read/write 方法自带 BE/LE 参数;DNS 解析、地址转换都有现成 API。HTTP 更是内置:http 模块与 fetch 全球通行,一个 createServer 就是生产级服务端的最小雏形。

必须保留的 C 式警觉是 协议设计 :TCP 是字节流,没有消息边界,"粘包/拆包"是永恒话题;C 里靠定长结构或长度前缀解决,Node 里做法完全一样——本章的协议实验会完整走一遍。

前置知识自查

- C 的 socket 编程:connect/bind/listen/accept/send/recv
- TCP 三次握手与"字节流无边界"的本质;UDP 数据报语义
- 端口、IP 地址、htons/htonl 字节序
- 第5章 Buffer 与第7章流的概念
- 第9章的事件循环(本章先用事件回调,不必深究机制)

🚀 本章学习路线

1. **第一阶段(概念与 TCP)**:读 §8.1–§8.2,跑通回声服务器与客户端
2. **第二阶段(UDP 与超时)**:读 §8.3–§8.4,理解无连接语义与超时必要性
3. **第三阶段(HTTP)**:读 §8.5–§8.6,用 fetch 与 createServer 打通请求-响应闭环
4. **第四阶段(协议设计)**:读 §8.7,完成章末"带长度前缀协议的文件传输"
5. **验收标准**:能独立写出回声服务器 + 客户端,并说明粘包为什么发生、长度前缀怎么解决

本章知识导图

- 第8章 网络通信
 - §8.1 概念回顾:C 的 socket API → Node 对照表
 - §8.2 TCP:net 模块服务端与客户端
 - §8.3 UDP:dgram 无连接通信
 - §8.4 超时与非阻塞:必须设置超时
 - §8.5 HTTP 客户端:fetch 全家桶
 - §8.6 HTTP 服务端:createServer 最小服务器
 - §8.7 生产坑:粘包/拆包、优雅关闭、进程退出

§8.1 概念回顾:从 C 的 socket 到 Node

C 的 API	Node 对应	说明
socket()	new net.Server() / new dgram.Socket()	创建即带类型,无需 AF_INET 参数
bind()	server.listen(port, host)	监听即绑定
listen()	server.listen(port, cb)	server.on('listening') 事件
accept()	server.on('connection', socket)	事件循环自动 accept,每连接一个 socket
connect()	net.connect(port, host)	客户端直连
send()/recv()	socket.write(data) / socket.on('data', cb)	全异步,自动缓冲
close()	socket.end() / socket.destroy()	end 优雅关闭(发完 FIN),destroy 立即断
htons/htonl	Buffer.writeUInt16BE/readUInt16LE 等	BE/LE 显式参数,无字节序暗坑
getaddrinfo	dns.promises.lookup()	域名解析(生产用系统缓存)

💡 直觉先行:Node 的 socket 是"事件 + 流"

net.Socket 既是可读流又是可写流:socket.write() 写出去,socket.on('data') 收进来。没有阻塞、没有返回码,错误走 'error' 事件,关闭走 'close'/'end' 事件。数据永远可能"半包到达"(流无边界),所以生产协议都要处理粘包——这是本章下半场的重点。

§8.2 TCP 编程:net 模块

8.2.1 回声服务器(对应 C 的 listen/accept 循环)

```
const net = require('node:net');

// 服务端:监听 8000 端口,收到什么回什么(回声)
const server = net.createServer((socket) => {
  console.log('新连接:', socket.remoteAddress, socket.remotePort);

  socket.on('data', (chunk) => {
    console.log('收到:', chunk.toString().trim());
    socket.write('echo: ' + chunk);    // 原样回写
  });
  socket.on('end', () => console.log('客户端断开(半关闭)'));
  socket.on('error', (err) => console.error('socket 错误:', err.message));
});

server.listen(8000, '127.0.0.1', () => {
  console.log('TCP 服务器监听 127.0.0.1:8000');
});

// 测试:node 另一个终端执行 node tcp_client.js,或 nc 127.0.0.1 8000
```

8.2.2 客户端(对应 C 的 connect/send/rcv)

```
const net = require('node:net');

const client = net.connect(8000, '127.0.0.1', () => {
  console.log('已连接服务器');
  client.write('你好,服务器!');
});

client.on('data', (chunk) => {
  console.log('服务器回复:', chunk.toString().trim());
  client.end();    // 优雅关闭(发送 FIN)
});

client.on('end', () => console.log('连接已关闭'));
client.on('error', (err) => console.error('客户端错误:', err.message));

// 发送多段数据:TCP 是流,不保证每次 write 对应一次 data(粘包伏笔)
```

⚠ 常见误区 8.1:以为一次 write 对应一次 data

TCP 是字节流,没有消息边界:客户端连续 write 三次,服务端可能收到一次或多次 data,数据还会被拆半(一个 write 的 10 字节可能分两次 data 到达)。千万不要按 data 次数解析协议,必须用定长/分隔符/长度前缀。第 8.7 节给出完整方案。

§8.3 UDP 编程:dgram 模块

UDP 是无连接数据报协议:不握手、不保证送达、不保证顺序,但有消息边界(一次 sendto 一个数据报)。适用场景:DNS 查询、音视频流、游戏状态、监控探针、广播发现。C 的 sendto/recvfrom 对应 dgram 的 send/message 事件。

```
const dgram = require('node:dgram');

// UDP 服务端
const udpServer = dgram.createSocket('udp4');
udpServer.on('message', (msg, rinfo) => {
  console.log(`收到 ${rinfo.address}:${rinfo.port} 的 ${msg.length} 字节`);
  const reply = Buffer.from('pong:' + msg.toString('utf8'));
  udpServer.send(reply, rinfo.port, rinfo.address); // 回给来源地址
});
udpServer.on('error', (err) => console.error('UDP 错误:', err.message));
udpServer.bind(8001, '127.0.0.1', () => console.log('UDP 监听 8001'));

// UDP 客户端:不需要连接,直接 send
const udpClient = dgram.createSocket('udp4');
const payload = Buffer.from('你好,UDP');
udpClient.send(payload, 8001, '127.0.0.1', (err) => {
  if (err) console.error('发送失败:', err.message);
});
udpClient.on('message', (msg) => {
  console.log('UDP 回复:', msg.toString('utf8'));
  udpClient.close(); // 用完关闭
});
```

§8.4 超时与非阻塞:为什么必须设置超时

⚠ 生产事故:不设超时的连接永久挂起

生产里最常见的网络事故之一:客户端连不上一台"黑洞"服务器(防火墙丢包、IP 不可达),connect 一直 pending,worker 线程被占满,最后整机雪崩。C 的 connect 有内核超时,Node 需要显式设置:每个出站连接与每次读操作都要有超时。

```

const net = require('node:net');

// 客户端超时:connect 超时 + 空闲超时
const client2 = net.connect({ port: 8002, host: '127.0.0.1', timeout: 2000 });
client2.on('timeout', () => {
  console.error('连接超时,销毁连接');
  client2.destroy(); // 超时必须主动销毁,否则挂着
});
client2.on('error', (err) => console.error(err.code === 'ETIMEDOUT' ? '超时失败' : err.message));

// 服务端空闲超时:清理僵尸连接
const server2 = net.createServer((socket) => {
  socket.setTimeout(10000); // 10 秒无数据
  socket.on('timeout', () => {
    console.log('空闲超时,断开:', socket.remoteAddress);
    socket.destroy();
  });
  socket.on('data', () => socket.setTimeout(10000)); // 有数据重置计时
});
server2.listen(8002, '127.0.0.1');

// HTTP 客户端超时:AbortSignal.timeout(现代 fetch)
async function fetchWithTimeout(url, ms = 5000) {
  const res = await fetch(url, { signal: AbortSignal.timeout(ms) });
  return res;
}
// fetchWithTimeout('http://127.0.0.1:8002/', 1000).catch((e) => console.log('超时:', e.name));

```

★ 重点结论:超时三原则

① 出站必设超时:connect timeout + idle timeout,超时即 destroy;② 入站必设空闲超时:防止慢客户端占着连接不放(慢速 DoS 的基本形态);③ 超时后必须显式清理资源,光报错不销毁等于没设。

§8.5 HTTP 客户端:fetch(浏览器与 Node 通用)

C 没有标准 HTTP 库,生产用 libcurl;Node 20+ 内置 `fetch` (与浏览器同一套标准 API),发 HTTP 请求不再需要第三方库。核心概念:Response 对象、await、状态码、响应体读取。

```

// GET 请求
async function getTodos() {
  const res = await fetch('https://jsonplaceholder.typicode.com/todos/1');
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const data = await res.json();      // 解析 JSON
  console.log('GET 结果:', data.title);
}
// getTodos().catch((e) => console.error(e));

// POST + JSON
async function createPost() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: '新文章', userId: 1 }),
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const created = await res.json();
  console.log('创建成功,ID:', created.id);
}

// 读文本与读二进制
async function readBoth() {
  const textRes = await fetch('https://example.com/');
  const text = await textRes.text();    // 文本响应
  console.log('文本长度:', text.length);

  const binRes = await fetch('https://example.com/');
  const buf = Buffer.from(await binRes.arrayBuffer()); // 二进制响应
  console.log('字节数:', buf.length);
}

// 统一超时 + 错误处理模板(生产必用)
async function safeFetch(url, options = {}) {
  const res = await fetch(url, {
    ...options,
    signal: options.signal ?? AbortSignal.timeout(10000),
  });
  if (!res.ok) {
    throw new Error(`请求失败:${res.status} ${res.statusText}`);
  }
  return res;
}

```

⚠ 常见误区 8.2:fetch 不抛错 = 成功?

fetch 只在网络层失败 (DNS、连接拒绝、超时)时 reject;HTTP 404/500 照常 resolve。必须检查 `res.ok` 或 `res.status`。另外响应体只能消费一次:读了 `json()` 就不能再读 `text()`,报错 "Body is unusable"。

§8.6 HTTP 服务端:http.createServer

最小 HTTP 服务器(生产框架 Express/Fastify 的底层就是它):

```

const http = require('node:http');

const server = http.createServer((req, res) => {
  // req:请求对象;res:响应对象
  const url = new URL(req.url, `http://${req.headers.host}`);

  if (req.method === 'GET' && url.pathname === '/hello') {
    res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('你好,世界');
    return;
  }

  if (req.method === 'POST' && url.pathname === '/api/echo') {
    let body = '';
    req.on('data', (chunk) => { body += chunk; });    // 异步收集请求体
    req.on('end', () => {
      try {
        const parsed = JSON.parse(body || '{}');
        res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
        res.end(JSON.stringify({ received: parsed, at: Date.now() }));
      } catch (err) {
        res.writeHead(400, { 'Content-Type': 'application/json; charset=utf-8' });
        res.end(JSON.stringify({ error: '非法 JSON' }));
      }
    });
    return;
  }

  res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
  res.end('Not Found');
});

server.listen(3000, '127.0.0.1', () => {
  console.log('HTTP 服务端:http://127.0.0.1:3000');
});
// 生产方向:Express(最流行)/Fastify(性能)/Koa(轻量) 都是它的封装

```

```

// 配合 fetch 测试上面服务器
async function testServer() {
  const r1 = await fetch('http://127.0.0.1:3000/hello');
  console.log('GET /hello:', r1.status, await r1.text());

  const r2 = await fetch('http://127.0.0.1:3000/api/echo', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name: '测试' }),
  });
  console.log('POST /api/echo:', r2.status, await r2.text());
}
testServer().catch((e) => console.error('测试失败:', e.message));

```

工程建议 8.1:请求体大小限制与响应头规范

生产服务端三件事:① 限制请求体大小(超限直接 413),防止恶意大包拖垮内存;② 统一 Content-Type 与 charset=utf-8,中文不乱码;③ 错误响应统一 JSON 结构({error, code}),方便前端统一处理。这些在 Express 里是中间件,在裸 http 里要自己写——知道原理即可。

§8.7 生产坑:粘包/拆包、优雅关闭

8.7.1 粘包与拆包:为什么发生,怎么解决

粘包:客户端两次 write 的数据被内核合并成一次 data 到达(收端看到"黏在一起");**拆包**:一次 write 的数据被拆成多次 data。根因:TCP 是字节流,收端只按"到达多少读多少"回调,不知道你的消息边界在哪。三种协议设计:

方案	原理	适用
定长	每条消息固定 N 字节,收满 N 处理一条	固定结构记录
分隔符	以 \n 或 \0 结尾,按分隔符切分	文本协议、HTTP 头部
长度前缀	头 4 字节存 payload 长度,按长度取整条	二进制协议(生产主流)


```

// 长度前缀协议:封装"编码"与"解码缓冲"
const HEADER_SIZE = 4;

// 编码:长度(4 字节 BE) + payload
function encodeMessage(payload) {
  const body = Buffer.from(payload, 'utf8');
  const buf = Buffer.alloc(HEADER_SIZE + body.length);
  buf.writeUInt32BE(body.length, 0);
  body.copy(buf, HEADER_SIZE);
  return buf;
}

// 解码:增量缓冲,从 data 流中切出完整消息(解决粘包/拆包)
class FrameParser {
  constructor() {
    this.buf = Buffer.alloc(0);
  }
  push(chunk) {
    this.buf = Buffer.concat([this.buf, chunk]);
    const messages = [];
    while (this.buf.length >= HEADER_SIZE) {
      const len = this.buf.readUInt32BE(0);
      if (this.buf.length < HEADER_SIZE + len) break; // 半包,等下一块
      const body = this.buf.subarray(HEADER_SIZE, HEADER_SIZE + len);
      messages.push(body.toString('utf8'));
      this.buf = this.buf.subarray(HEADER_SIZE + len); // 消费掉整条
    }
    return messages;
  }
}

// 使用:
const parser = new FrameParser();
parser.push(Buffer.concat([encodeMessage('第一条'), encodeMessage('第二条')]]));
// ↑ 上面一次推入两条,模拟粘包
console.log(parser.push(encodeMessage('第三条'))); // 第三条
// 拆包模拟:分两次推
const full = encodeMessage('大消息');
parser.push(full.subarray(0, 5)); // 只推头部+1字节
console.log('半包时输出:', parser.push(full.subarray(5)).length); // 仍是 0 条
// 补齐剩余字节后:

```

8.7.2 优雅关闭与进程退出

```
// 优雅停机:停止接新连接 → 处理完存量 → 退出
function gracefulShutdown(server, signal) {
  console.log(`收到 ${signal},开始优雅停机`);
  server.close(() => {
    console.log('所有连接已关闭,进程退出');
    process.exit(0);
  });
  // 兜底:超时强制退出(防呆)
  setTimeout(() => {
    console.error('强制退出');
    process.exit(1);
  }, 10000).unref(); // unref:不影响进程自然退出
}

// 挂到信号(生产容器/PM2 都会发 SIGTERM)
process.on('SIGTERM', () => gracefulShutdown(server, 'SIGTERM'));
process.on('SIGINT', () => gracefulShutdown(server, 'SIGINT'));

// TIME_WAIT 说明:服务端 close 后端口进入 TIME_WAIT(与 C 相同);
// 生产高并发重启时监听失败,常见对策:SO_REUSEADDR(内核默认)或等待,Node 一般无需处理
```

⚠ 常见误区 8.3:进程退出前不清理连接

直接 `process.exit()` 会粗暴掐断所有连接,客户端收到 `ECONNRESET` 而不是正常 `FIN`,前端可能重试风暴。正确顺序:`server.close()` 拒绝新连接 → 等待存量请求完成 → 超时兜底强制退出。Docker/K8s 的 `SIGTERM` 就是触发这个流程的标准化信号。

本章速查表

主题	速查要点
TCP 服务端	net.createServer(cb) + listen(port);connection 事件给 socket
TCP 客户端	net.connect(port, host);write 发送,data 接收
socket 事件	data/end(半关闭)/close/error/timeout
关闭语义	end() 优雅(FIN);destroy() 立即断(ECONNRESET)
UDP	dgram.createSocket('udp4') + send/bind;无连接、有消息边界
超时	connect timeout、socket.setTimeout、AbortSignal.timeout(ms)
超时处理	超时后必须 destroy,否则连接永久挂着
fetch GET	fetch(url) → res.ok 检查 → res.json()/text()/arrayBuffer()
fetch POST	method/headers/body 选项;JSON 用 stringify
fetch 的坑	HTTP 错误不 reject,必须查 res.ok;body 只能消费一次
HTTP 服务端	http.createServer((req, res));res.writeHead + res.end
请求体	req 是流:on('data') 收集 + on('end') 处理
粘包/拆包	TCP 是字节流;定长/分隔符/长度前缀三种协议
长度前缀	4 字节 BE 长度 + payload;增量缓冲切帧
优雅停机	server.close() → 等存量完成 → 超时兜底;处理 SIGTERM/SIGINT

最小必会清单

- 能写出 TCP 回声服务器与客户端,并说出 5 个 socket 事件
- 能说出 end() 与 destroy() 的区别(优雅 vs 立即)
- 能写出 UDP 收发两端,并说出 UDP 与 TCP 的三个差异
- 能写出"连接超时 + 空闲超时 + 销毁"三件套
- 能用 fetch 完成 GET/POST/JSON,并检查 res.ok
- 能用 createServer 写出路由分发(路径 + 方法)
- 能解释粘包/拆包根因,并说出三种协议方案
- 能写出 FrameParser 增量切帧(核心代码)
- 能写出优雅停机流程与信号处理
- 能说出 fetch 与 createServer 在生产框架中的位置(Express 等是封装)

自测题

- Q** 1. TCP 客户端连续 socket.write('A'); socket.write('B'),服务端一定收到两次 data 吗?

✓ 参考答案

不一定。TCP 是字节流,可能合并成一次 'AB',也可能拆成多次。协议解析不能依赖 data 次数,要用定长/分隔符/长度前缀。

Q 2. `socket.end()` 与 `socket.destroy()` 的区别是什么?

✓ 参考答案

`end()` 发送 FIN 优雅关闭,缓冲数据写完再关,对端收到 'end' 事件;`destroy()` 立即销毁连接,对端收到 ECONNRESET。

Q 3. `fetch` 请求一个返回 500 的接口,会抛异常吗?为什么?如何正确判断?

✓ 参考答案

不会。`fetch` 只在网络层失败时 reject,HTTP 状态码错误照常 resolve。用 `res.ok(200-299)`或 `res.status` 判断。

Q 4. 为什么生产代码必须给每个出站连接设置超时?超时后该做什么?

✓ 参考答案

不设超时,连接可能永久 pending(黑洞 IP、防火墙丢包),耗尽连接池/线程导致雪崩。超时后必须 `destroy/abort` 清理资源,不能只报错。

Q 5. 简述长度前缀协议的编码与解码两个环节,以及解码时的"半包"怎么处理。

✓ 参考答案

编码:4 字节 BE 长度 + payload。解码:维护增量缓冲,先读长度,缓冲不足就等下一块 data;足够就切出整条并消费,循环处理粘包的多条。

Q 6. 服务端收到 SIGTERM 后应该按什么顺序做什么?

✓ 参考答案

`server.close()` 停止接受新连接 → 等待存量请求自然结束(close 回调)→ 设置超时兜底强制退出 → 退出进程。顺序不能反。

章末实验题:带协议的文件传输服务器

实验 8:带长度前缀协议的文件传输(ftans)

实验目标:实现"请求-传输-确认"的 TCP 文件传输:客户端请求文件,服务端按长度前缀协议分块发送,客户端组装并校验,覆盖全章知识点。

实验要求:

- 任务 1(覆盖:TCP 服务端/客户端):net 双端,服务端按文件名找文件,找不到回复错误帧
- 任务 2(覆盖:长度前缀协议、粘包/拆包):消息帧 = 4 字节类型/长度 + payload;客户端用 FrameParser 增量切帧;故意把两次发送合并模拟粘包
- 任务 3(覆盖:超时):客户端 connect 超时与空闲超时,超时销毁并报错;服务端空闲超时清理连接
- 任务 4(覆盖:Buffer、二进制):文件内容用 Buffer 传输;分块(每块 64KB)避免大文件一次写爆内存;接收端按 Buffer.concat 组装
- 任务 5(覆盖:HTTP/fetch 辅助):附赠一个 fetch 健康检查端点(服务端同时起 http 端口返回状态 JSON)
- 任务 6(覆盖:优雅关闭):SIGINT/SIGTERM 优雅停机;server.close + 超时兜底
- 任务 7(覆盖:错误处理):文件不存在(ENOENT)返回错误帧;客户端校验字节数并打印

实验环境:Node.js 20+, 两个终端: `node ftans.js server 8000 /tmp/share` 与 `node ftans.js client 127.0.0.1 8000 hello.txt`。

第一步 定义协议:帧 = type(1 字节:1=文件头,2=数据块,3=完成,4=错误)+ len(4 字节 BE)+ payload;头 5 字节

第二步 实现双端状态机:服务端按文件流式发帧;客户端按 FrameParser 收帧组装

第三步 后加健壮性:超时、优雅关闭、错误帧、字节数校验,打印传输统计

✅ 参考答案要点

核心:5 字节帧头 + 增量切帧 + 流式分块传输 + 双端超时与优雅关闭。约 160 行,覆盖全章知识点。

```
// ftrans.js — 运行:node ftrans.js server 8000 /tmp/share | node ftrans.js client 127.0.0.1 8000 hell
o.txt
'use strict';
const net = require('node:net');
const http = require('node:http');
const fs = require('node:fs');
const path = require('node:path');

const HEADER = 5; // 1 字节 type + 4 字节 len
const TYPE = { HEAD: 1, DATA: 2, DONE: 3, ERROR: 4 };
const CHUNK = 64 * 1024;

// 1. 编码帧(覆盖:Buffer 写入、字节序)
function frame(type, payload) {
  const body = payload ?? Buffer.alloc(0);
  const buf = Buffer.alloc(HEADER + body.length);
  buf.writeUInt8(type, 0);
  buf.writeUInt32BE(body.length, 1);
  body.copy(buf, HEADER);
  return buf;
}

// 2. 增量切帧器(覆盖:长度前缀协议、粘包/拆包)
class FrameParser {
  constructor() { this.buf = Buffer.alloc(0); }
  push(chunk) {
    this.buf = Buffer.concat([this.buf, chunk]);
    const frames = [];
    while (this.buf.length >= HEADER) {
      const type = this.buf.readUInt8(0);
      const len = this.buf.readUInt32BE(1);
      if (this.buf.length < HEADER + len) break; // 半包等待
      frames.push({ type, body: this.buf.subarray(HEADER, HEADER + len) });
      this.buf = this.buf.subarray(HEADER + len);
    }
    return frames;
  }
}

// 3. 服务端(覆盖:createServer、流式读文件、错误帧)
function startServer(port, shareDir) {
  const server = net.createServer((socket) => {
    socket.setTimeout(30000);
    socket.on('timeout', () => { console.log('服务端:空闲超时,断开'); socket.destroy(); });
    const parser = new FrameParser();
    socket.on('data', (chunk) => {
      for (const f of parser.push(chunk)) {
        if (f.type !== TYPE.HEAD) continue;
        const name = f.body.toString('utf8');
        const file = path.join(shareDir, path.basename(name)); // 防路径穿越
        if (!fs.existsSync(file)) {
          socket.write(frame(TYPE.ERROR, Buffer.from('文件不存在:' + name, 'utf8')));
          continue;
        }
        const size = fs.statSync(file).size;

```

```

        socket.write(frame(TYPE.HEAD, Buffer.from(JSON.stringify({ name, size })), 'utf8')));
        const rs = fs.createReadStream(file, { highWaterMark: CHUNK });
        rs.on('data', (chunkData) => socket.write(frame(TYPE.DATA, chunkData)));
        rs.on('end', () => socket.write(frame(TYPE.DONE, Buffer.alloc(0))));
        rs.on('error', (err) => socket.write(frame(TYPE.ERROR, Buffer.from(err.message, 'utf8'))));
    }
});
socket.on('error', (e) => console.error('连接错误:', e.message));
});
server.listen(port, '127.0.0.1', () => console.log(`文件服务监听 ${port}, 目录 ${shareDir}`));

// 附赠 HTTP 健康检查(覆盖:createServer、JSON 响应)
http.createServer((req, res) => {
    res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
    res.end(JSON.stringify({ ok: true, uptime: process.uptime() }));
}).listen(port + 1);

graceful(server, '文件服务');
return server;
}

// 4. 客户端(覆盖:connect、超时、组装、校验)
function startClient(host, port, name, outPath) {
    const socket = net.connect({ host, port, timeout: 3000 });
    const parser = new FrameParser();
    let meta = null;
    const parts = [];

    socket.on('connect', () => socket.write(frame(TYPE.HEAD, Buffer.from(name, 'utf8'))));

    socket.on('data', (chunk) => {
        for (const f of parser.push(chunk)) {
            if (f.type === TYPE.ERROR) {
                console.error('服务器错误:', f.body.toString('utf8'));
                socket.destroy();
                return;
            }
            if (f.type === TYPE.HEAD) {
                meta = JSON.parse(f.body.toString('utf8'));
                console.log(`开始接收 ${meta.name}, 大小 ${meta.size} 字节`);
            } else if (f.type === TYPE.DATA) {
                parts.push(Buffer.from(f.body)); // 组装(可优化为写入流)
            } else if (f.type === TYPE.DONE) {
                const out = Buffer.concat(parts);
                fs.writeFileSync(outPath, out);
                console.log(`接收完成:${out.length} 字节, 校验${out.length === meta.size ? '通过' : '失败!'}`);
                socket.end();
            }
        }
    });
    socket.on('timeout', () => { console.error('客户端:连接/空闲超时'); socket.destroy(); });
    socket.on('error', (err) => console.error('客户端错误:', err.message));
}

// 5. 优雅关闭(覆盖:信号、close、超时兜底)

```



```

function graceful(server, tag) {
  function shutdown(sig) {
    console.log(`${tag}收到 ${sig},优雅停机`);
    server.close(() => {
      console.log('连接已清空,退出');
      process.exit(0);
    });
    setTimeout(() => { console.error('兜底强制退出'); process.exit(1); }, 5000).unref();
  }
  process.on('SIGTERM', () => shutdown('SIGTERM'));
  process.on('SIGINT', () => shutdown('SIGINT'));
}

// 6. 入口(覆盖:参数解析、默认值)
function main() {
  const [mode, a, b, c] = process.argv.slice(2);
  if (mode === 'server') {
    const dir = c ?? b ?? '/tmp/share';
    fs.mkdirSync(dir, { recursive: true });
    startServer(Number(a ?? 8000), dir);
  } else if (mode === 'client') {
    const host = a ?? '127.0.0.1';
    const port = Number(b ?? 8000);
    const name = c ?? 'hello.txt';
    startClient(host, port, name, '/tmp/download-' + name);
  } else {
    console.error('用法:node ftrans.js server <端口> <目录> | node ftrans.js client <host> <端口> <文件名>');
    process.exitCode = 1;
  }
}
main();

```

设计说明:协议帧 type+len+payload 演示长度前缀方案的完整落地;FrameParser 同时处理粘包(服务端连续 write 的帧可能合并到达)与半包(大文件分块后单块被 TCP 拆散);服务端用 createReadStream 按 64KB 分块,内存占用与文件大小无关;客户端 Buffer.concat 组装后按 meta.size 校验;双端超时与优雅关闭按生产标准实现。测试建议:先写一个 2MB 测试文件,再用 `node ftrans.js client 127.0.0.1 8000 测试文件.bin` 传输,对比字节数与文件大小。

下一章预告:第9章 并发与异常 —— 事件循环的宏任务/微任务、Promise/async/await 全解、setTimeout 顺序题,以及 throw 与 Promise 错误传播——JS 最核心的一章。